

Multi-Tier AI Face Recognition Attendance Hub with Edge Inference and Manual CSV Sync Management

Submitted by: Bhushan Z.
Roll No: 134 Class: TYCS-B

July 17, 2026

1 Introduction & Abstract

Modern attendance management requires balancing automation speed with administrative accuracy. This project introduces a distributed, full-stack facial recognition framework that separates student self-enrollment, local teacher edge verification, and manual validation loops. A decentralized web dashboard allows students to upload their face data remotely to a centralized database.

The teacher's localized management application pulls down these pre-computed 128-dimensional face signatures to run recognition edge inference locally via classroom camera feeds. The system auto-compiles an offline CSV attendance ledger, allowing teachers to audit poor-lighting anomalies before executing an intentional manual sync back to the primary SQL enterprise database.

2 Problem Statement

Current automated attendance tracking applications suffer from two primary execution flaws:

1. **High Server Overhead:** Streaming high-definition classroom video to a centralized cloud server for heavy convolutional neural network (CNN) parsing introduces extreme network latency and server resource exhaustion.
2. **Rigid Automated Biases:** Committing raw machine learning assumptions directly to a production database leaves zero error-handling room for environmental false negatives (e.g., poor lighting, obstructed angles), resulting in faulty academic records.

3 Proposed Solution & Methodology

This platform provides a low-latency, hardware-agnostic solution divided into four operational pipelines:

- **Phase 1: Student Enrollment:** Students use the HTML5 MediaStream API to capture facial coordinates, passing vectors through an open extractor model to upload a 128-dimensional mathematical array as a serialized text blob.
- **Phase 2: Local Cache Sync:** The teacher’s application fetches active classroom rosters and caches the embedding strings into local instance memory, bypassing real-time server network bottlenecks.
- **Phase 3: Edge Recognition & CSV Export:** A camera snapshot scans student profiles in parallel using Euclidean distance calculation rules via OpenCV and dlib. Unverified exceptions are written to a localized spreadsheet file.
- **Phase 4: Manual SQL Commit:** The teacher audits and uploads the edited .csv file. The backend runs data validation and parses rows into production SQL tables via a database upsert rule, preventing double-entries.

4 Technical Stack

Table 1: System Implementation Technologies

Component	Technology
Web Portal Engine	HTML5 Canvas API, CSS3 Grid, Modern JavaScript (Fetch REST)
Application Framework	FastAPI / Flask (Python Ecosystem)
Computer Vision Layer	OpenCV, Face Recognition Framework (dlib baseline model)
Relational Data Storage	SQLite / PostgreSQL (Mapped JSON text tracking logic)
Data Structuring Core	NumPy matrix computation tools & Pandas IO engines

5 Expected Outcomes

- **Decentralized Architecture:** A working biometric pipeline isolated from central network constraints via edge inference.
- **Sub-millisecond Edge Matching:** Rapid local facial array comparisons across classroom groups against cached matrix pools.
- **Human-in-the-Loop Sync:** A secure, transaction-safe update pipeline that parses teacher CSV modifications while strictly enforcing database constraints.